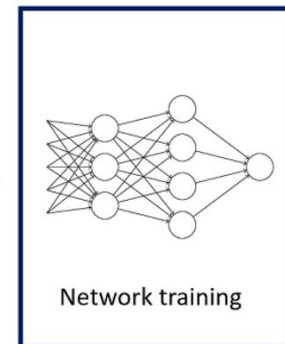


CNN- MNIST-demo

Dr. Grantham Pang

Dept. of Elec. & Computer Engg.,
The University of Hong Kong

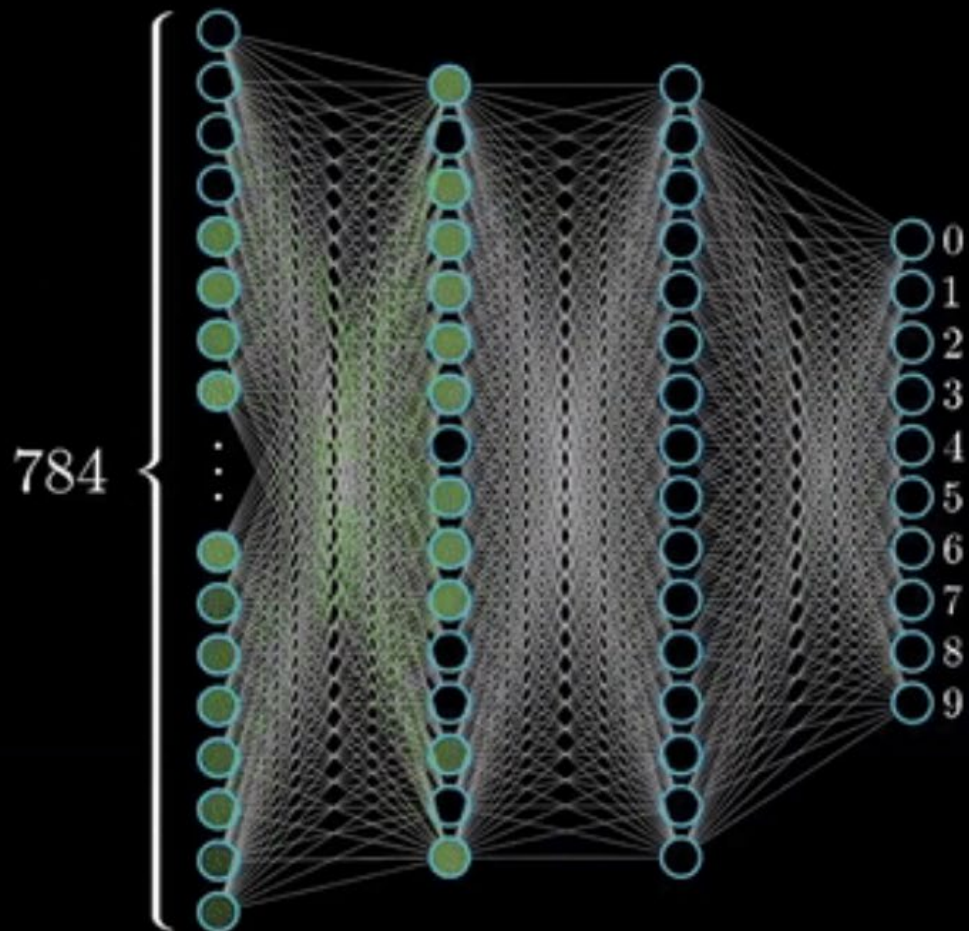
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
1 1 1 1 1 1 1 1 1 1 1 1 1 1
2 2 2 2 2 2 2 2 2 2 2 2 2 2
3 3 3 3 3 3 3 3 3 3 3 3 3 3
4 4 4 4 4 4 4 4 4 4 4 4 4 4
5 5 5 5 5 5 5 5 5 5 5 5 5 5
6 6 6 6 6 6 6 6 6 6 6 6 6 6
7 7 7 7 7 7 7 7 7 7 7 7 7 7
8 8 8 8 8 8 8 8 8 8 8 8 8 8
9 9 9 9 9 9 9 9 9 9 9 9 9 9



Data & Labels

0
1
2
3
4
5
6
7
8
9

Example 1: (1D) 784 inputs



The file `Class04D2_MNIST.ipynb`

EXAMPLE 1: typical ANN

The example based on 784 (28x28)

inputs, 1D input, **900 hidden nodes**

Baseline Error: 1.53% 25 epochs

Total params: 715510

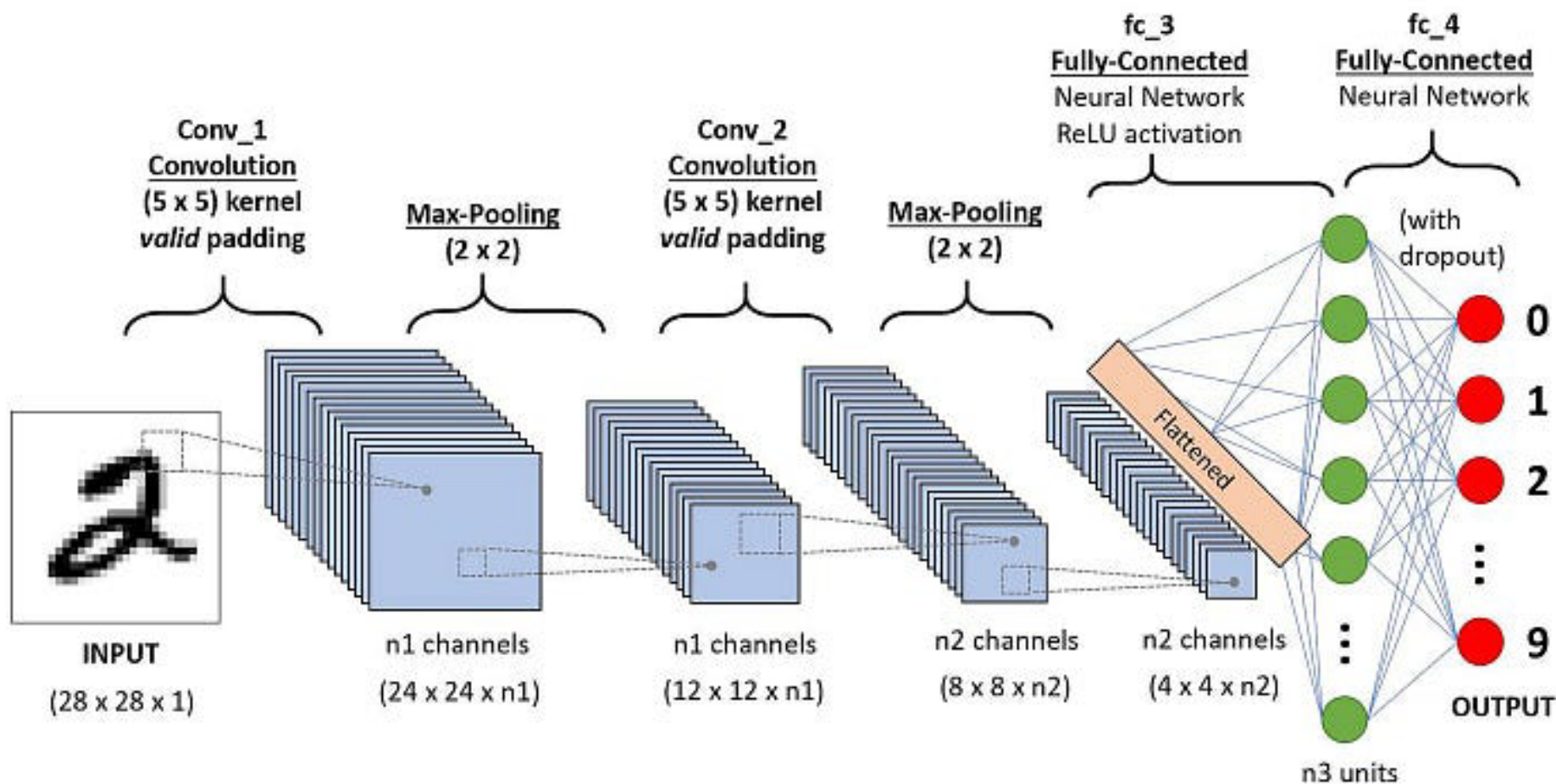
EXAMPLE 2:

The example based on a simple CNN

2D input, **32 (5x5) filters, 128 nodes**

CNN Error: 1.2% 10 epochs

Total params: 592074



In artificial intelligence and machine learning, Convolutional Neural Networks (**CNNs**) have emerged as a groundbreaking tool, especially in the domain of **image processing and analysis**. CNNs are particularly designed to automatically and adaptively learn spatial hierarchies of features from input images.

The file `Class04D2_MNIST.ipynb`

EXAMPLE 2 (version 2):

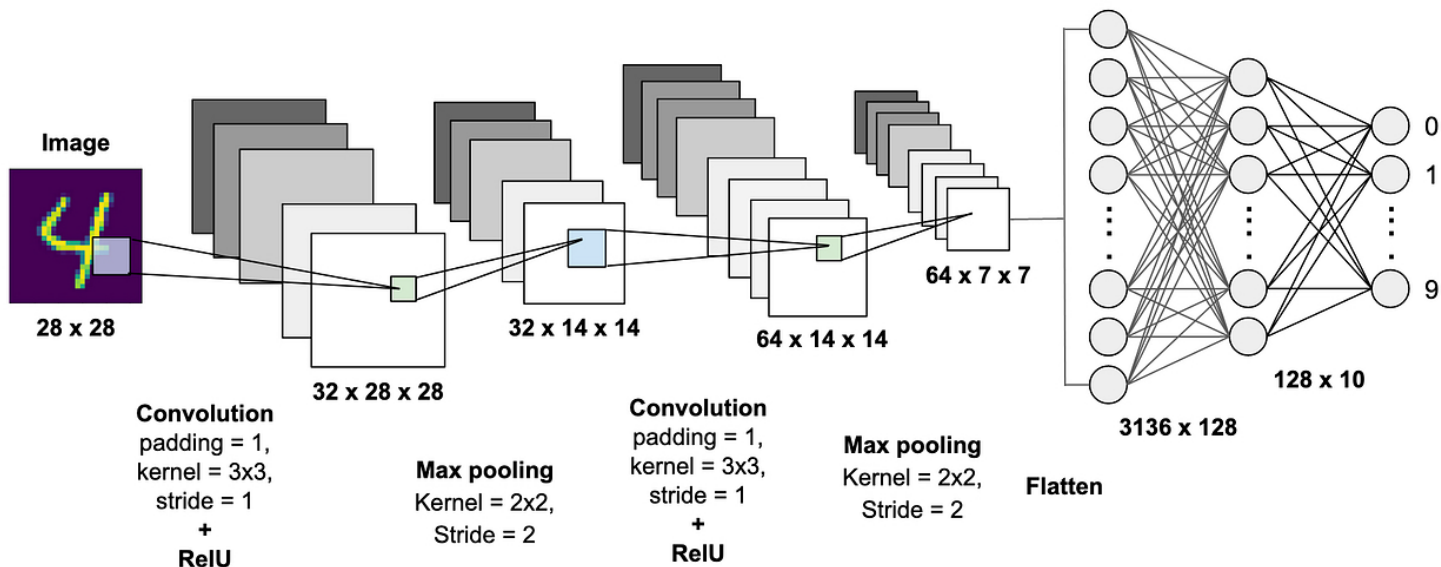
The example based on a simple CNN

2D input, **60 (5x5) filters, 64 nodes**

with **Dropout(..)**

CNN Error: 1.07% 30 epochs

Total params: 555234



EXAMPLE 3:

The example based on another simple CNN structure,

2D input, **64 (3x3) filters, 32 (3x3) filters ; 128 nodes**

with **Dropout(..)**

CNN Error: 0.86% 10 epochs

Total params: 610346

EXAMPLE 4:

The example based on a larger CNN structure,

2D input, **128 (3x3) filters, 128 (3x3) filters ;**

Two fully connected layers: **128 nodes; 50 nodes;**

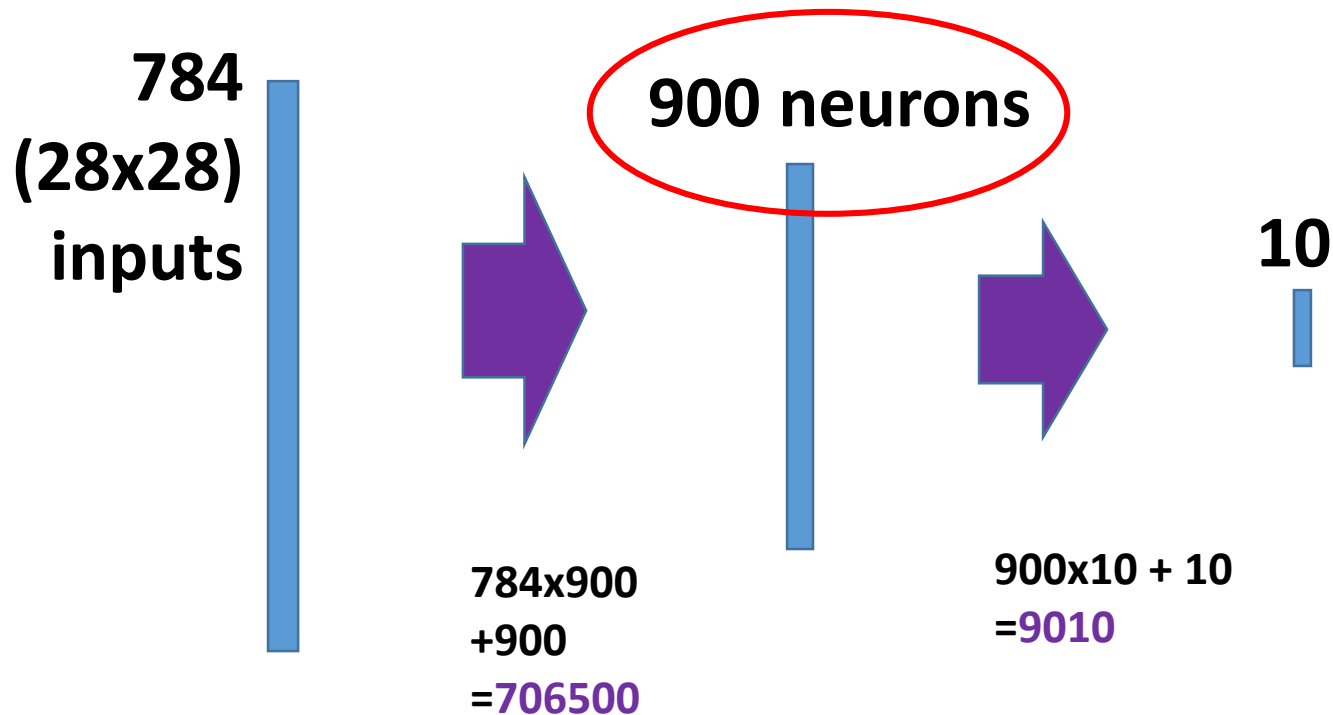
with **Dropout(0.2)**

Large CNN Error: 0.98% 10 epochs

Total params: 2515248

Class04D2_MNIST.ipynb

Example 1: ANN



Total number of parameters = $706500 + 9010 = 715510$

```
mode = 'jupyter' # mode = 'colab' or 'jupyter'
if mode == 'colab':
    from google.colab import drive
    drive.mount('/content/drive')
```

```
if mode == 'jupyter':
    # Important libraries for running in jupyter notebook
    print('import library for jupyter notebook')
    import numpy as np
    import matplotlib.pyplot as plt
    from tensorflow.keras.datasets import mnist
    from tensorflow.keras.models import Sequential
    from tensorflow.keras.layers import Dense
    from tensorflow.keras.layers import Dropout
    from tensorflow.keras.layers import Flatten
    # from keras.layers.convolutional import Conv2D #OLD
    from tensorflow.keras.layers import Conv2D
    from tensorflow.keras.layers import MaxPooling2D
    # from keras.utils import np_utils #OLD
    from tensorflow.keras.utils import to_categorical
    from tensorflow.keras import backend as K
```

```
import library for jupyter notebook
```

Slight difference
between running in
colab and jupyter
notebook

Example from the MNIST database

```
1 import keras
2 from keras.datasets import mnist
3
4 (x_train,y_train),(x_test,y_test)=mnist.load_data()
5
6 print('x_train.shape:',x_train.shape)
7 print('y_train.shape:',y_train.shape)
8 print('x_test.shape:',x_test.shape)
9 print('y_test.shape:',y_test.shape)
10
11 aa=x_train[0]
12 print('shape of x_train[0]:',aa.shape)
```

```
x_train.shape: (60000, 28, 28)
y_train.shape: (60000,)
x_test.shape: (10000, 28, 28)
y_test.shape: (10000,)
shape of x_train[0]: (28, 28)
```



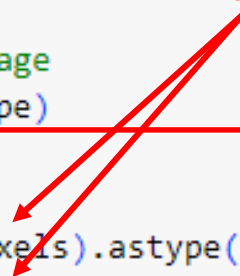
```
# EXAMPLE 1:  
# ANN example based on 784 (28x28) inputs, 1D input  
#  
from keras.datasets import mnist
```

```
# load the MNIST dataset  
(X_train, y_train), (X_test, y_test) = mnist.load_data()  
  
# fix random seed for reproducibility  
seed = 7  
np.random.seed(seed)
```

```
#  
# flatten 28*28 images to a 784 vector for each image  
print('X_train.shape before reshape' , X_train.shape)  
num_pixels = X_train.shape[1] * X_train.shape[2]  
  
X_train = X_train.reshape(X_train.shape[0], num_pixels).astype('float32')  
X_test = X_test.reshape(X_test.shape[0], num_pixels).astype('float32')  
  
print('X_train.shape after reshape' , X_train.shape)
```

```
# normalize inputs from 0-255 to 0-1  
X_train = X_train / 255  
X_test = X_test / 255  
# one hot encode outputs  
if mode == 'colab':  
    y_train = to_categorical(y_train)  
    y_test = to_categorical(y_test)
```

784



```
num_classes = y_test.shape[1]
num_hidden_nodes = 900
```

```
# define baseline model
# Dense : a regular fully connected layer
# The chosen compile model has a loss function, and the purpose is to compute the quantity that
# the model should minimize during training. Below, the 'categorical_crossentropy' is used as
# the loss function. (The MSE is not chosen to be the loss function.) It would compute the crossentropy
# loss between the labels and predictions. There are two or more label classes, usually with labels
# provided in a one_hot representation.
```

```
def baseline_model():
    # create model
    model = Sequential()
    model.add(Dense(num_hidden_nodes, input_dim=num_pixels, kernel_initializer='normal', activation='relu'))
    model.add(Dense(num_classes, kernel_initializer='normal', activation='softmax'))
    # Compile model
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
    # Choice of optimizer: adam (adaptive moment estimation), AdaGrad (adaptive learning rate),
    # sgd (Stochastic gradient descent), RMSprop (similar to AdaGrad), Adadelta (adaptive delta) ...
    return model

# build the model
model = baseline_model()

# Fit the model
# verbose : number of steps before showing a result
history = model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=25, batch_size=200, verbose=2)
# Final evaluation of the model
scores = model.evaluate(X_test, y_test, verbose=0)
```

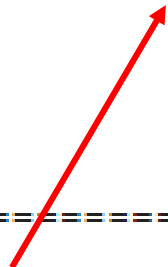
784

```
[24] model.summary()
```

```
Model: "sequential_3"
```

Layer (type)	Output Shape	Param #
dense_6 (Dense)	(None, 900)	706500
dense_7 (Dense)	(None, 10)	9010
Total params: 715510 (2.73 MB)		
Trainable params: 715510 (2.73 MB)		
Non-trainable params: 0 (0.00 Byte)		

with 900 hidden nodes



Calculation of the number of parameters:

$$784 \times 900 + 900 = 706500$$

$$900 \times 10 + 10 = 9010$$

```
# EXAMPLE 2:
# The example based on a simple CNN structure, 2D input
#

# define baseline model
def baseline_model():
    # create model
    model = Sequential()
    model.add(Conv2D(32, (5, 5), input_shape=(28, 28, 1), activation='relu'))
    model.add(MaxPooling2D(pool_size=(2, 2)))
    model.add(Dropout(0.2))
    model.add(Flatten())
    model.add(Dense(128, activation='relu'))
    model.add(Dense(num_classes, activation='softmax'))
    # Compile model
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model

#
# build the model
model = baseline_model()
model.summary()

# Fit the model
model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=10, batch_size=200, verbose=2)

# Final evaluation of the model

scores = model.evaluate(X_test, y_test, verbose=0)
print("CNN Error: %.2f%%" % (100-scores[1]*100))
```

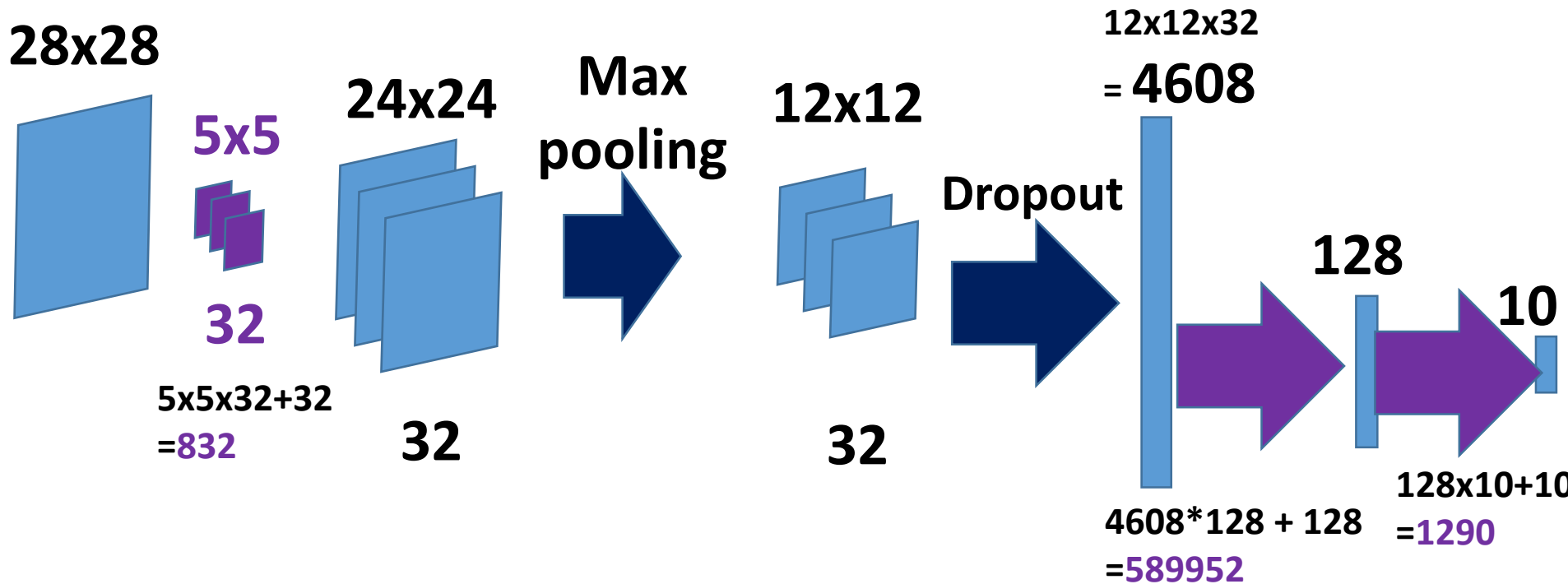
Layer (type)	Output Shape	Param #
conv2d_6 (Conv2D)	(None, 24, 24, 32)	832
max_pooling2d_6 (MaxPooling2D)	(None, 12, 12, 32)	0
dropout_6 (Dropout)	(None, 12, 12, 32)	0
flatten_6 (Flatten)	(None, 4608)	0
dense_20 (Dense)	(None, 128)	589952
dense_21 (Dense)	(None, 10)	1290

=====
 Total params: 592074 (2.26 MB)
 Trainable params: 592074 (2.26 MB)
 Non-trainable params: 0 (0.00 Byte)

explanation
 in the next page

Class04D2_MNIST.ipynb

Example 2: simple CNN # 2D input, 60 (5x5) filters, 64 nodes



Total number of parameters = $832 + 589952 + 1290 = 592074$

Epoch 10/10

300/300 - 1s - loss: 0.0160 - accuracy: 0.9947 - val_loss: 0.0341 - val_accuracy: 0.9894 - 1s/epoch
CNN Error: 1.06%

EXAMPLE 2 (another version):

The example based on a simple CNN structure, 2D input
#

define baseline model

def baseline_model():

create model

model = Sequential()

model.add(Conv2D(60, (5, 5), input_shape=(28, 28, 1), activation='relu'))

model.add(MaxPooling2D(pool_size=(2, 2)))

model.add(Dropout(0.3))

model.add(Flatten())

model.add(Dense(64, activation='relu'))

model.add(Dense(num_classes, activation='softmax'))

Compile model

model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])

return model

#

build the model

model = baseline_model()

model.summary()

Fit the model

60 (5x5) filters, 64 nodes

Epoch 30/30

300/300 - 1s - loss: 0.0044 - accuracy: 0.9985 - val_loss: 0.0439 - val_accuracy: 0.9903
CNN model error: 0.97%

Layer (type)	Output Shape	Param #
conv2d_7 (Conv2D)	(None, 24, 24, 60)	1560
max_pooling2d_7 (MaxPooling2D)	(None, 12, 12, 60)	0
dropout_7 (Dropout)	(None, 12, 12, 60)	0
flatten_7 (Flatten)	(None, 8640)	0
dense_22 (Dense)	(None, 64)	553024
dense_23 (Dense)	(None, 10)	650

```

=====
Total params: 555234 (2.12 MB)
Trainable params: 555234 (2.12 MB)
Non-trainable params: 0 (0.00 Byte)

```

EXAMPLE 3:

The example based on another simple CNN structure, 2D input

#

```
def baseline_model():
```

```
    # create model
```

```
    model = Sequential()
```

64 (3x3) filters, 32 (3x3) filters ; 128 nodes

```
    model.add(Conv2D(64, kernel_size=(3, 3), input_shape=(28, 28, 1), activation='relu'))
```

```
    model.add(Conv2D(32, kernel_size=(3, 3), activation='relu'))
```

```
    model.add(MaxPooling2D(pool_size=(2, 2)))
```

```
    model.add(Dropout(0.25))
```

```
    model.add(Flatten())
```

```
    model.add(Dense(128, activation='relu'))
```

```
    model.add(Dropout(0.5))
```

```
    model.add(Dense(num_classes, activation='softmax'))
```

```
    # Compile model
```

```
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
```

```
    return model
```

```
model = baseline_model()
```

```
model.summary()
```

Layer (type)	Output Shape	Param #
conv2d_26 (Conv2D)	(None, 26, 26, 64)	640
conv2d_27 (Conv2D)	(None, 24, 24, 32)	18464
max_pooling2d_20 (MaxPooling2D)	(None, 12, 12, 32)	0
dropout_17 (Dropout)	(None, 12, 12, 32)	0
flatten_17 (Flatten)	(None, 4608)	0
dense_50 (Dense)	(None, 128)	589952
dropout_18 (Dropout)	(None, 128)	0
dense_51 (Dense)	(None, 10)	1290

**How do we
get these
number of
parameters
?**

```

Total params: 610346 (2.33 MB)
Trainable params: 610346 (2.33 MB)
Non-trainable params: 0 (0.00 Byte)

```

300/300 - 3s - loss: 0.0310 - accuracy: 0.9899 - val_loss: 0.0284 - val_accuracy: 0.9906 - 3s/epoch - 12ms/step
CNN Error: 0.94%

```
# EXAMPLE 4:
# The example based on a larger CNN structure, 2D input
#
# define the larger model
from timeit import default_timer as timer

def larger_model():
    # create model
    model = Sequential()
    model.add(Conv2D(128, (3, 3), input_shape=(28, 28, 1), activation='relu'))
    # model.add(MaxPooling2D(pool_size=(2, 2)))
    model.add(Conv2D(128, (3, 3), activation='relu'))
    model.add(MaxPooling2D(pool_size=(2, 2)))
    model.add(Dropout(0.2))
    model.add(Flatten())
    model.add(Dense(128, activation='relu'))
    model.add(Dropout(0.2))
    model.add(Dense(50, activation='relu'))
    model.add(Dense(num_classes, activation='softmax'))
    # Compile model
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model
```

128 (3x3) filters, 128 (3x3) filters

128 nodes; 50 nodes

Layer (type)	Output Shape	Param #
conv2d_24 (Conv2D)	(None, 26, 26, 128)	1280
conv2d_25 (Conv2D)	(None, 24, 24, 128)	147584
max_pooling2d_19 (MaxPooling2D)	(None, 12, 12, 128)	0
dropout_15 (Dropout)	(None, 12, 12, 128)	0
flatten_16 (Flatten)	(None, 18432)	0
dense_47 (Dense)	(None, 128)	2359424
dropout_16 (Dropout)	(None, 128)	0
dense_48 (Dense)	(None, 50)	6450
dense_49 (Dense)	(None, 10)	510

```

=====
Total params: 2515248 (9.59 MB)
Trainable params: 2515248 (9.59 MB)
Non-trainable params: 0 (0.00 Byte)

```

Epoch 10/10

300/300 [=====] - 9s 30ms/step - loss: 0.0111 - accuracy: 0.9963 -

Execution time = 143.4338635490003

Large CNN Error: 0.93%